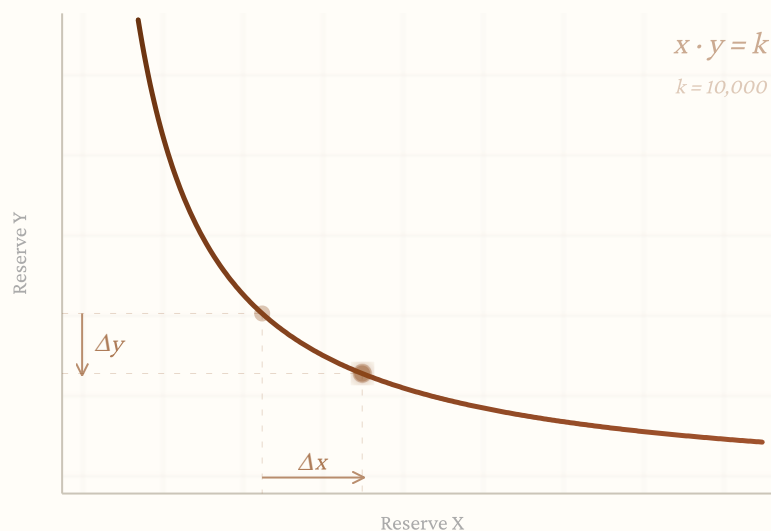


About the Competition

Designed by [Benedict Brady](#) and [Dan Robinson](#)

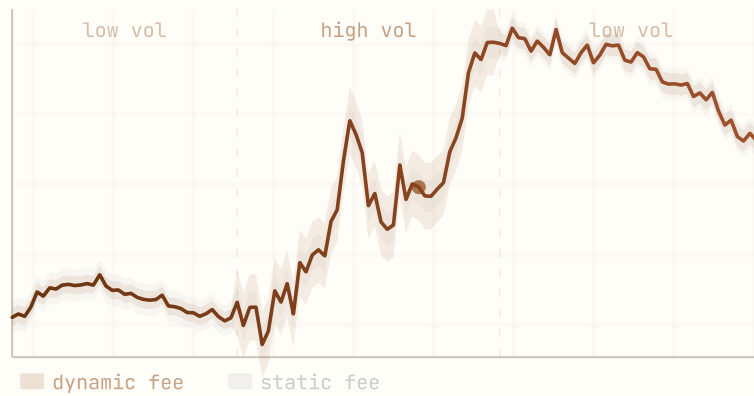
What is an AMM?

An Automated Market Maker (AMM) is a smart contract that holds reserves of two tokens and allows users to trade between them. The most common design uses the $x \cdot y = k$ constant product formula, where the product of reserves stays constant after each trade. AMMs charge fees on trades, which accumulate as profit for liquidity providers.



The Challenge

Your goal is to design a fee strategy that maximizes profitability. Static fees (like Uniswap V2's 30 bps) leave money on the table—sometimes fees are too low and you lose to arbitrageurs, other times they're too high and you lose volume to competitors. Additionally, you don't know the volatility of the price or the arrival rate of retail orders—your strategy has to deduce those from the market. A well-designed dynamic fee strategy can adapt to market conditions and capture more value.

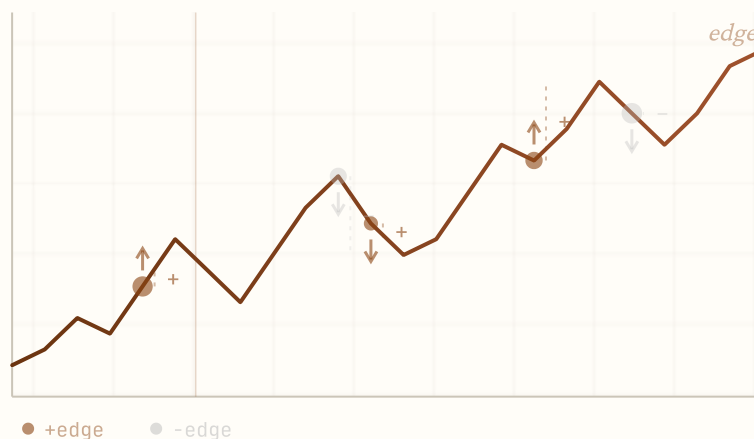


How Scoring Works

Strategies are scored by **edge**—for each trade, how much you made or lost compared to the true price of the asset (the price that arbitrageurs trade to). If the AMM sells X above this price or buys X below it, that's positive edge.

$$\begin{aligned} \text{Edge} = & \sum (\text{amount}_x \times \text{true_price} - \text{amount}_y) && \text{for sells (AMM sells X)} \\ & + \sum (\text{amount}_y - \text{amount}_x \times \text{true_price}) && \text{for buys (AMM buys X)} \end{aligned}$$

Retail trades generate positive edge (you profit from the spread). Arbitrage trades generate negative edge (informed flow extracts value). Your total edge is aggregated across 1000 randomized simulations with varying market conditions.



The Simulation

Each simulation runs 10,000 steps with a price process and retail order flow. Before each simulation, hyperparameters (volatility, arrival rate, order size) are sampled randomly from a range—your strategy doesn't know these values and must adapt from observed trades.

Price Process (GBM)

The true market price follows Geometric Brownian Motion:

$$S(t+1) = S(t) \cdot \exp(-\sigma^2/2 + \sigma Z)$$

where $Z \sim N(0,1)$

Drift $\mu = 0$ (no directional bias). Per-step volatility σ is sampled uniformly from [0.088%, 0.101%] at the start of each simulation and held fixed throughout.



Retail Order Flow

Uninformed traders arrive via Poisson process. Each simulation samples its own arrival rate and order size distribution:

```
arrivals ~ Poisson( $\lambda$ )           $\lambda \sim U[0.6, 1.0]$  per sim
size ~ LogNormal( $\mu$ ,  $\sigma=1.2$ )  mean  $\sim U[19, 21]$  in Y terms
direction = buy with prob 0.5, sell with prob 0.5
```

Order Routing

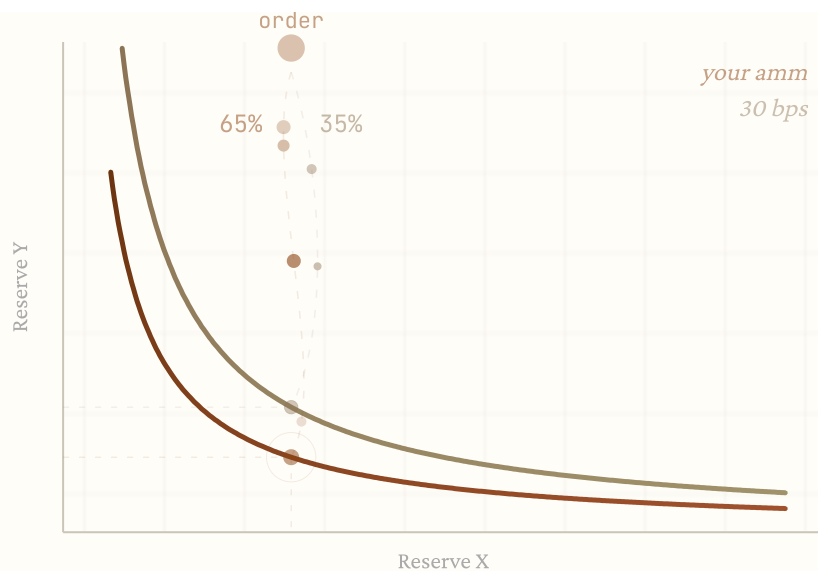
Orders are split optimally between the two AMMs to equalize marginal execution prices.

For a buy order of size Y split between AMM_1 and AMM_2 :

```
 $y_i = 1 - fee_i$ 
 $A_i = \sqrt{(x_i * y_i * y_i)}$ 
 $r = A_1 / A_2$ 

 $\Delta y_1 = (r * (y_2 + y_2 * Y) - y_1) / (y_1 + r * y_2)$ 
```

Flow capture depends on both your fee and your current reserves. Higher fees or a less favorable mid price reduce your share of incoming orders.



Why There's Another AMM

Your strategy shares the market with a default 30bps AMM. This second AMM acts as a **normalizer**—it prevents degenerate strategies from appearing profitable. If your fees are too high, retail flow routes to the other AMM and you earn nothing. If your fees are too low, you take bad trades. The goal is to maximize your own edge, not to “beat” the other AMM.

Writing a Strategy

Strategies are written in Solidity and must extend `AMMStrategyBase`. Strategies work somewhat similarly to Uniswap V4 hooks—they have functions that are called at certain points in the lifecycle of the AMM. You implement two such functions:

afterInitialize

Called once at the start of each simulation with the AMM's starting reserves. Return your initial buy fee (`bidFee`) and sell fee (`askFee`).

```
function afterInitialize(uint256 initialX, uint256 initialY)
    external returns (uint256 bidFee, uint256 askFee)
```

afterSwap

Called after every trade that hits your AMM. You receive details about the trade that just happened, and return the fees you want to show the market going forward. Note that this function does not affect the fee for the current trade.

```
function afterSwap(TradeInfo calldata trade)
    external returns (uint256 bidFee, uint256 askFee)
```

```
// TradeInfo contains:
//   bool    isBuy      - true if AMM bought X (trader sold X)
//   uint256 amountX    - amount of X traded
//   uint256 amountY    - amount of Y traded
//   uint256 timestamp   - simulation step number
//   uint256 reserveX    - post-trade X reserves
//   uint256 reserveY    - post-trade Y reserves
```

Fees are in WAD precision ($1e18 = 100\%$). Use the helper `bpsToWad(30)` for 30 basis points. You have 32 storage slots (`slots[0] – slots[31]`) to track state between trades.

Example: Spread After Big Trades

A simple strategy that widens fees after large trades and decays back to a base fee over time:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AMMStrategyBase} from "./AMMStrategyBase.sol";
import {TradeInfo} from "./IAMMStrategy.sol";

contract Strategy is AMMStrategyBase {
    function afterInitialize(uint256, uint256)
        external override returns (uint256, uint256)
    {
        return (bpsToWad(30), bpsToWad(30));
    }

    function afterSwap(TradeInfo calldata trade)
        external override returns (uint256, uint256)
    {
        // slots[0] = current fee in bps (WAD-encoded)
        uint256 fee = slots[0];
        if (fee == 0) fee = bpsToWad(30);

        // If trade was large relative to reserves, widen the fee
        uint256 tradeRatio = wdiv(trade.amountY, trade.reserveY);
        if (tradeRatio > WAD / 20) { // > 5% of reserves
            fee = clampFee(fee + bpsToWad(10));
        } else {
            // Decay back toward 30 bps
            uint256 base = bpsToWad(30);
            if (fee > base) fee = fee - bpsToWad(1);
        }

        slots[0] = fee;
        return (fee, fee);
    }

    function getName() external pure override
        returns (string memory)
```

```
{  
    return "Spread After Big Trades";  
}
```

Rules & Constraints

- Gas limit: 250,000 gas for both `afterInitialize()` and `afterSwap()`
- Storage limit: 32 storage slots (1KB)
- No external calls or oracles
- No assembly or inline Yul
- Fees must be between 0 and 1,000 basis points (10%)

Submit Your Strategy

 Leaderboard

 Submit

 GitHub

 About

Benedict Brady 

Dan Robinson 